Business Understanding & DBMS Foundations

Financial Risk and Performance Analysis in Consumer Lending

Company Name: CRED

Date: 12-02-2026

Product Dissection

1. Platform Selection

Question: Choose a leading platform from a domain related to the FinTech/Banking industry. Justify your selection by discussing the platform's popularity, impact, and relevance in its industry.

Answer: CRED

Popularity

CRED is one of India's most prominent premium FinTech platforms, specifically focusing on credit card users with high credit scores (750+). Its popularity is driven by a unique value proposition that rewards users for paying their credit card bills on time, combined with a single platform for all credit info of the user and strong brand identity. Through word-of-mouth, strategic marketing, and exclusive membership criteria, CRED has built a loyal and highly engaged user community.

Impact

Traditionally, credit card bill payments were seen as a routine financial obligation with no immediate benefit. CRED transformed this behavior by incentivizing timely repayments through rewards, thereby encouraging financial discipline. This shift has benefited not only users by helping them maintain strong credit scores but also banks and financial institutions, which experience improved repayment rates and reduced default risk. CRED's data-driven approach has also demonstrated how technology can be leveraged to influence positive financial behavior at scale.

Relevance

In an era where digital payments and credit usage are rapidly increasing, CRED with its focus on creditworthiness, timely repayments, and responsible financial behavior aligns closely with the core objectives of the banking and lending industry. Furthermore, CRED's expansion into financial services such as lending, pay later products, and partner-based credit offerings makes it a comprehensive FinTech platform. By combining payments, rewards, and credit products, CRED plays a crucial role in bridging consumer convenience with financial responsibility.

2. Core Features and Functionalities

Question: Research and list the core features and functionalities of the selected platform. Describe how these features contribute to the platform's success and user engagement.

Answer: Core Features and Functionalities of CRED

- **Credit Card Bill Payments:**
  Allows users to pay credit card bills from multiple banks in one place, simplifying financial management and improving user convenience.
- **Rewards System (CRED Coins):**
  Rewards users for timely bill payments, encouraging repeat usage and reinforcing positive financial behavior.
- **Credit Score–Focused Membership:**
  Restricts access to users with good credit scores, creating a premium user base with lower financial risk and higher spending power.
- **Payment Reminders and Notifications:**
  Helps users avoid missed payments, reducing penalties and protecting their credit scores.
- **CRED Store and Brand Offers:**
  Provides exclusive deals, discounts, and offers from partner brands, increasing engagement and perceived value for users.
- **Lending and Credit Products:**
  Offers short-term loans, credit lines, and "buy now, pay later" options through partner institutions, expanding CRED's role beyond payments.
- **User-Friendly Interface:**
  A clean and intuitive app design enhances usability and encourages frequent interaction.
- **Data-Driven Personalization:**
  Uses user behavior and transaction patterns to provide tailored offers and financial products.

These features collectively drive CRED's growth, engagement, and strategic positioning in the FinTech ecosystem.

3. Real World Problems

Question: Identify the real-world problems that the platform aims to solve. Discuss how the platform addresses these problems through its features and functionalities.

Answer: Real-World Problems Addressed by CRED

1. **Missed Credit Card Payments**
   - **Problem:** Many users forget due dates, leading to late fees, penalties, and reduced credit scores.
   - **Solution:** CRED provides timely reminders and a centralized payment system, ensuring users pay their bills on time.
2. **Lack of Incentives for Responsible Credit Behavior**
   - **Problem:** Paying credit card bills on time traditionally offers no immediate reward to users.
   - Solution: CRED rewards users with CRED coins for timely payments, turning responsible behavior into a rewarding experience.
3. **Fragmented Credit Card Management**
   - **Problem:** Users with multiple credit cards struggle to track bills across different banks and platforms.
   - **Solution:** CRED consolidates all credit card bills into a single platform, simplifying management and improving financial organization.
4. **Limited Access to Tailored Financial Products**
   - **Problem:** Consumers often receive generic financial offers that may not match their credit profile.

- **Solution:** By analyzing repayment behavior and creditworthiness, CRED offers personalized financial products such as loans and pay-later options.

5. **Low Financial Awareness and Discipline**
    - **Problem:** Many consumers lack visibility into how their repayment behavior affects their financial health.
    - **Solution:** CRED promotes awareness through reminders, insights, and rewards, encouraging disciplined financial habits.

How CRED Addresses These Problems

- Centralized Payments & Reminders: Prevent missed payments and penalties.
- Rewards System: Encourages disciplined repayment behavior.
- Credit-Based Membership: Ensures a reliable and low-risk user base.
- Personalized Financial Products: Improves relevance and user satisfaction.
- Clean UX and Transparency: Builds trust and promotes financial awareness.

Together, these features enable CRED to solve key real-world challenges in consumer credit management while enhancing user engagement and financial responsibility.

Schema Design

users

Field Name	Data Type	Description
user_id	BIGSERIAL (PK)	Unique identifier for each user
full_name	VARCHAR(100)	Full name of the user
email	VARCHAR(100)	User's email address (unique)
phone	VARCHAR(15)	User's mobile number (unique)
status	VARCHAR(20)	User account status (ACTIVE, INACTIVE, BLOCKED)
created_at	TIMESTAMP	Account creation timestamp

user_kyc

Field Name	Data Type	Description
kyc_id	BIGSERIAL (PK)	Unique KYC record identifier
user_id	BIGINT (FK)	Reference to the user
kyc_type	VARCHAR(30)	Type of KYC (Aadhaar, PAN, Passport)
kyc_number_hash	TEXT	Hashed KYC identifier
verified_status	VARCHAR(20)	KYC status (PENDING, VERIFIED, REJECTED)
verified_at	TIMESTAMP	KYC verification timestamp

kyc_documents

Field Name	Data Type	Description
doc_id	BIGSERIAL (PK)	Unique document identifier
kyc_id	BIGINT (FK)	Reference to KYC record
doc_type	VARCHAR(30)	Document type (ID proof, address proof)

storage_uri	TEXT	Secure storage location of document
uploaded_at	TIMESTAMP	Document upload timestamp

accounts

Field Name	Data Type	Description
account_id	BIGSERIAL (PK)	Unique account identifier
user_id	BIGINT (FK)	Owner of the account
account_type	VARCHAR(30)	Savings, Wallet, Credit
currency	VARCHAR(10)	Currency code (INR, USD)
status	VARCHAR(20)	Account status
created_at	TIMESTAMP	Account creation time

account_balances

Field Name	Data Type	Description
account_id	BIGINT (PK, FK)	Reference to account
available_balance	NUMERIC(15,2)	Spendable balance
ledger_balance	NUMERIC(15,2)	Balance including pending txns
updated_at	TIMESTAMP	Last balance update time
Field Name	Data Type	Description
txn_id	BIGSERIAL (PK)	Unique transaction ID
user_id	BIGINT (FK)	User initiating transaction
txn_type	VARCHAR(40)	Payment, Transfer, Repayment
amount	NUMERIC(15,2)	Transaction amount
currency	VARCHAR(10)	Transaction currency
status	VARCHAR(20)	SUCCESS, FAILED, PENDING

initiated_at	TIMESTAMP	Transaction start time
completed_at	TIMESTAMP	Transaction completion time

transactions

Field Name	Data Type	Description
ledger_entry_id	BIGSERIAL (PK)	Unique ledger entry
account_id	BIGINT (FK)	Affected account
txn_id	BIGINT (FK)	Associated transaction
entry_type	VARCHAR(10)	DEBIT or CREDIT
amount	NUMERIC(15,2)	Entry amount
balance_after	NUMERIC(15,2)	Account balance after entry
created_at	TIMESTAMP	Entry creation time

ledger_entries

Field Name	Data Type	Description
card_id	BIGSERIAL (PK)	Unique credit card ID
user_id	BIGINT (FK)	Card owner
network	VARCHAR(20)	Visa, Mastercard, RuPay
last4	CHAR(4)	Last 4 digits of card
billing_cycle_day	INT	Monthly billing cycle day
status	VARCHAR(20)	ACTIVE, BLOCKED

credit_cards

Field Name	Data Type	Description
bill_id	BIGSERIAL (PK)	Credit card bill ID

card_id	BIGINT (FK)	Associated credit card
bill_month	VARCHAR(7)	Billing month (YYYY-MM)
total_due	NUMERIC(15,2)	Total bill amount
due_date	DATE	Payment due date
status	VARCHAR(20)	PAID, UNPAID, OVERDUE

loans

Field Name	Data Type	Description
loan_id	BIGSERIAL (PK)	Unique loan ID
user_id	BIGINT (FK)	Borrower
principal	NUMERIC(15,2)	Loan principal amount
interest_rate	NUMERIC(5,2)	Annual interest rate
tenure_months	INT	Loan duration
status	VARCHAR(20)	ACTIVE, CLOSED, DEFAULTED
disbursed_txn_id	BIGINT (FK)	Disbursement transaction

loan_repayments

Field Name	Data Type	Description
repayment_id	BIGSERIAL (PK)	Repayment record ID
loan_id	BIGINT (FK)	Reference to loan
due_date	DATE	Repayment due date
paid_date	DATE	Actual payment date
amount_due	NUMERIC(15,2)	Amount to be paid
amount_paid	NUMERIC(15,2)	Amount actually paid
status	VARCHAR(20)	PAID, PENDING, OVERDUE

fraud_alerts

Field Name	Data Type	Description
alert_id	BIGSERIAL (PK)	Fraud alert ID
txn_id	BIGINT (FK)	Related transaction
user_id	BIGINT (FK)	User involved
risk_score	NUMERIC(5,2)	Fraud risk score
status	VARCHAR(20)	OPEN, RESOLVED
created_at	TIMESTAMP	Alert creation time

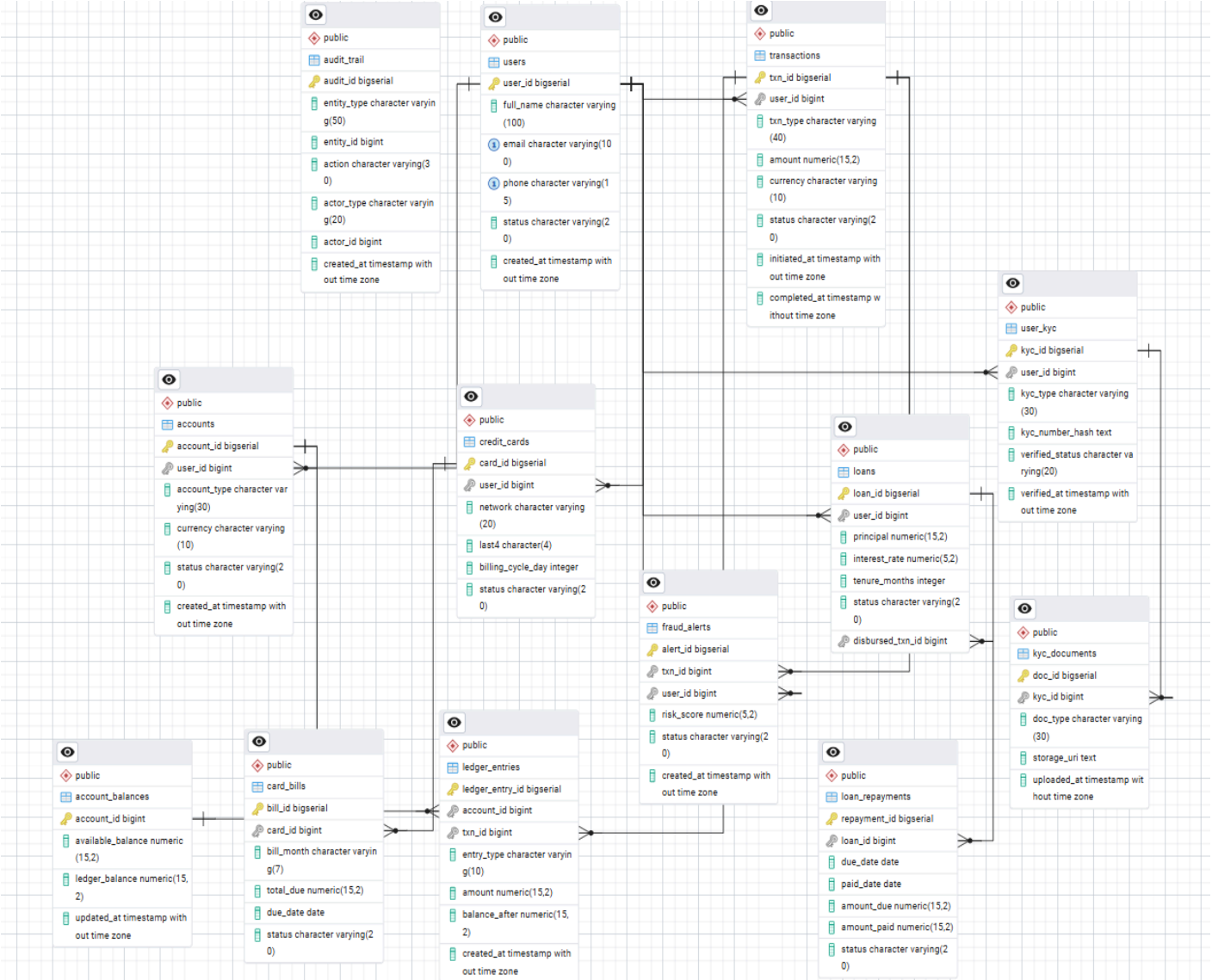
audit_trail

Field Name	Data Type	Description
audit_id	BIGSERIAL (PK)	Audit record ID
entity_type	VARCHAR(50)	Entity name (Loan, Account)
entity_id	BIGINT	Affected entity ID
action	VARCHAR(30)	CREATE, UPDATE, DELETE
actor_type	VARCHAR(20)	USER, SYSTEM, ADMIN
actor_id	BIGINT	Who performed the action
created_at	TIMESTAMP	Action timestamp

Relationships

From Table	From Attribute	To Table	To Attribute	Relationship Type	Explanation
user_kyc	user_id	users	user_id	Many-to-One	One user can have multiple KYC records
kyc_documents	kyc_id	user_kyc	kyc_id	Many-to-One	One KYC can have multiple documents
accounts	user_id	users	user_id	Many-to-One	One user can hold multiple accounts
account_balances	account_id	accounts	account_id	One-to-One	Each account has a single balance record
transactions	user_id	users	user_id	Many-to-One	A user can perform many transactions
ledger_entries	account_id	accounts	account_id	Many-to-One	An account has many ledger entries
ledger_entries	txn_id	transactions	txn_id	Many-to-One	A transaction creates multiple ledger entries
credit_cards	user_id	users	user_id	Many-to-One	A user can own multiple credit cards
card_bills	card_id	credit_cards	card_id	Many-to-One	Credit cards generate monthly bills
loans	user_id	users	user_id	Many-to-One	A user can take multiple loans
loan_repayments	loan_id	loans	loan_id	Many-to-One	A loan has multiple repayment schedules
loans	disbursed_txn_id	transactions	txn_id	One-to-One	Loan disbursement happens via one transaction
fraud_alerts	txn_id	transactions	txn_id	Many-to-One	A transaction may trigger fraud alerts
fraud_alerts	user_id	users	user_id	Many-to-One	Fraud alerts are associated with users
audit_trail	entity_id	Any Entity	Primary Key	Polymorphic	Tracks changes across all tables

ER Diagram


public
audit_trail
audit_id bigserial
entity_type character varying(50)
entity_id bigint
action character varying(30)
actor_type character varying(20)
actor_id bigint
created_at timestamp without time zone
public
users
user_id bigserial
full_name character varying(100)
email character varying(100)
phone character varying(15)
status character varying(20)
created_at timestamp without time zone
public
transactions
txn_id bigserial
user_id bigint
txn_type character varying(40)
amount numeric(15,2)
currency character varying(10)
status character varying(20)
initiated_at timestamp without time zone
completed_at timestamp without time zone
public
user_kyc
kyc_id bigserial
user_id bigint
kyc_type character varying(30)
kyc_number_hash text
verified_status character varying(20)
verified_at timestamp without time zone
public
accounts
account_id bigserial
user_id bigint
account_type character varying(30)
currency character varying(10)
status character varying(20)
created_at timestamp without time zone
public
credit_cards
card_id bigserial
user_id bigint
network character varying(20)
last4 character(4)
billing_cycle_day integer
status character varying(20)
public
loans
loan_id bigserial
user_id bigint
principal numeric(15,2)
interest_rate numeric(5,2)
tenure_months integer
status character varying(20)
disbursed_txn_id bigint
public
fraud_alerts
alert_id bigserial
txn_id bigint
user_id bigint
risk_score numeric(5,2)
status character varying(20)
created_at timestamp without time zone
public
kyc_documents
doc_id bigserial
kyc_id bigint
doc_type character varying(30)
storage_uri text
uploaded_at timestamp without time zone
public
account_balances
account_id bigint
available_balance numeric(15,2)
ledger_balance numeric(15,2)
updated_at timestamp without time zone
public
card_bills
bill_id bigserial
card_id bigint
bill_month character varying(7)
total_due numeric(15,2)
due_date date
status character varying(20)
public
ledger_entries
ledger_entry_id bigserial
account_id bigint
txn_id bigint
entry_type character varying(10)
amount numeric(15,2)
balance_after numeric(15,2)
created_at timestamp without time zone
public
loan_repayments
repayment_id bigserial
loan_id bigint
due_date date
paid_date date
amount_due numeric(15,2)
amount_paid numeric(15,2)
status character varying(20)

Case Study: Revenue and Profit Growth Strategies

To solve scenario-based questions such as increasing CRED's profit by 25%, data science students should adopt an inside-out methodology. This approach starts by analysing internal company data expenses, customer behaviour, and revenue streams before proposing optimization strategies

I. Revenue & Profit Growth Strategy (Inside-Out Methodology) – CRED

Category	Details
Current Financial Data	Collect Financial Data: Revenue: Card bill payments, instant credit facilitation, merchant-funded offers, rent payments, platform service charges. Expenses: Reward issuance, partner incentives, customer acquisition, technology operations, workforce, credit risk provisioning.
	Analyse Current Profit: Profit Baseline: Evaluate contribution margins at transaction-level and user-level. Profit Trends: Analyse margin trends across payments, credit, and partnerships to identify sustainable vs subsidised growth.
Sources of Revenue	Identify Revenue Streams: Payments (card bills, rent), credit products, brand collaborations, merchant commissions, platform fees.
	Analyse Revenue Contribution: Product-level revenue analysis across payments, credit, and offers. Identify high-revenue user segments and merchant categories driving disproportionate value.
Sources of Expenses	Identify Major Expenses: Reward costs, marketing & acquisition spend, credit losses, technology infrastructure, operational workforce.
	Analyse Expense Distribution: Reward ROI evaluation to assess repeat usage vs one-time subsidies. Risk cost analysis to isolate high-default segments and servicing overheads.
Customer Acquisition & Retention	Customer Behaviour Analysis: Frequency of bill payments, reward redemption, credit adoption, rent payments.
	Retention & Churn Analysis: Identify lifecycle drop-off points. Use transaction frequency and repayment behaviour to predict churn and lifetime value.

II. Focus Areas for Increasing CRED's Profit by 25%

To increase CRED's profit by 25%, the platform must strategically focus on several key areas aligned with its FinTech business model. These focus areas include internal management, product strategy, market expansion, post-transaction management, and branding & growth marketing. By addressing these areas with targeted, data-driven initiatives, CRED can improve operational efficiency, enhance monetization, and strengthen long-term user engagement.

Category-wise Focus Areas

Category	Focus Area	Measures
Internal Management (~8%)	Operational Efficiency (~6%)	Reward Cost Optimisation using transaction-level ROI analysis. Automate reconciliation, fraud checks, and customer support via AI workflows. Control costs through audits of cloud infra, APIs, and vendor contracts.
	Employee Productivity (~2%)	Upskill teams in data-driven decision-making, risk analytics, and automation. Introduce KPIs tied to revenue contribution, risk reduction, and efficiency.
Product Strategy (~6%)	New Product Expansion (~3%)	Expand credit-led products (instant credit, BNPL, EMI) for high-credit-score users. Launch premium subscriptions offering enhanced rewards, insights, and concierge services.
	Product Optimisation (~3%)	Prune low-margin or low-usage features using data-led analysis. Bundle payments, rewards, and credit to increase ARPU.
Market Expansion (~5%)	User Segment Expansion (~3%)	Carefully onboard near-prime users using improved risk models. Integrate with banks, fin-techs, and lifestyle platforms.
	Market Penetration (~2%)	Drive higher transaction frequency via rent payments, UPI usage, recurring bills. Cross-sell financial products using behavioural insights.
Post-Transaction Management (~3%)	Customer Satisfaction (~1.5%)	Implement continuous feedback loops. Faster, transparent dispute resolution for payments and credit.
	Customer Retention (~1.5%)	Tier-based loyalty programs rewarding repeat usage and timely repayments. Personalised nudges and offers based on user behaviour.
Branding & Growth Marketing (~3%)	Brand Positioning (~1%)	Reinforce premium brand image via curated partnerships and experiences. Financial education and lifestyle content to build trust.
	Referrals & Partnerships (~2%)	Incentivise high-value users to refer similar profiles. Strategic alliances with luxury, travel, and fintech brands.

III. Defining Strategies

Category	Details
Optimise Expenses	Cost Reduction: Optimise reward distribution, renegotiate third-party contracts, and improve cloud capacity planning. Strengthen credit risk management to reduce loan defaults.

	Efficiency Improvements: Automate reconciliation, fraud detection, and customer support. Optimise transaction processing and reward issuance systems.
Enhance Revenue Streams	Upselling & Cross-Selling: Encourage bill-payment users to adopt rent payments, instant credit, and partner offers through personalised recommendations. New Revenue Streams: Launch premium subscriptions, expand lending products, and introduce data-driven merchant monetisation. Pricing Strategies: Apply risk-based pricing for credit and performance-linked merchant commissions.
Improve Customer Satisfaction & Retention	Personalised Experiences: Use transaction behaviour, credit data, and engagement signals to deliver tailored rewards, reminders, and insights. Loyalty Programs: Strengthen tier-based programs rewarding consistency, repayment discipline, and referrals. Customer Feedback: Continuously analyse feedback to improve usability, transparency, and trust.

Guesstimates

1. What will be the percentage increase in global FinTech investments over the next five years?

Step	Details	Assumption / Calculation
Current Global FinTech Investment	Approx. global annual investment today	~$200 Billion
Growth Drivers	Digital payments, AI risk models, embedded finance	Strong
Annual Growth Rate (Assumed)	Conservative CAGR	~10% per year
5-Year Growth Factor	(1.10)^5	≈ 1.61
Future Investment	200 × 1.61	≈ $322 Billion
Percentage Increase	(322 − 200) / 200	~60% increase

Final Estimate: Global FinTech investments are expected to increase by ~60% over the next 5 years

2. How many people will adopt digital banking services in developing countries over the next decade?

Approach: Demand-side population breakdown

Step	Details	Calculation
Population (Developing Countries)	Asia, Africa, LATAM	~6 Billion
Adult Population (15+)	65%	3.9 Billion
Current Digital Banking Users	~40% penetration	1.6 Billion
Internet Growth Impact	Smartphone + 5G adoption	+30%
New Users in 10 Years	3.9 × 0.30	~1.17 Billion
Rounding	Conservative estimate	~1.2 Billion

Final Estimate: 1.2 Billion new users will adopt digital banking in developing countries over the next decade

3. What percentage of small and medium-sized enterprises (SMEs) will use FinTech solutions for their financial needs by 2025?

Approach: SME adoption funnel

Step	Details	Assumption
Total SMEs (Global)	Approximation	~400 Million
SMEs with Internet Access	70%	280 Million
SMEs Using Digital Payments	60%	168 Million
SMEs Using FinTech Lending / Tools	50%	84 Million
Adoption Percentage	84 / 168	~50%

Final Estimate: 50% of SMEs will use FinTech solutions for financial needs by 2025

4. What will be the average transaction value of mobile payments in the next three years?

Approach: Usage behavior + inflation-adjusted growth

Step	Details	Value
Current Avg Mobile Transaction	Wallets & UPI	~$25
Usage Shift	Small → medium value transactions	Increasing
Annual Increase	Due to trust & merchant adoption	~10%
3-Year Growth	(1.10)^3	~1.33
Future Avg Transaction Value	25 × 1.33	~$33

Final Estimate: Average mobile payment transaction value will be ~$30–35 in the next 3 years

5. How much will blockchain technology reduce the costs of cross-border transactions in the next five years?

Approach: Cost comparison (Traditional vs Blockchain)

Step	Details	Cost
Traditional Cross-Border Fee	Banks & intermediaries	6–8%
Blockchain-Based Fee	Stablecoins / DLT rails	1–2%
Average Cost Reduction	6% → 2%	~4%
Percentage Reduction	(4 / 6) × 100	~65%
Conservative Adjustment	Regulatory & adoption delays	~50–60%

Final Estimate: Blockchain can reduce cross-border transaction costs by ~50–60% over the next 5 years

A/B Testing

Scenario 1:

A fintech company offers a mobile payment app where users can link their bank accounts and make payments. The company wants to analyse the retention of users who signed up for the app in the past six months to understand how often they continue using it after the initial download.

Question 1: How would you perform a cohort analysis to calculate the monthly retention rate for users who signed up in different months?

To understand how frequently users continue using a mobile payment app after signing up, a cohort analysis can be performed. This method groups users based on their signup period and tracks their activity over time to measure retention patterns.

1. Objective of the Cohort Analysis

The primary objective is to analyze monthly user retention by identifying how often users who signed up in different months continue to make transactions in subsequent months. This helps assess user engagement, product stickiness, and long-term platform usage.

2. Define the Cohort

A cohort is a group of users who share a common characteristic—in this case, the month of signup.

Example Cohorts:

- January Signup Cohort
- February Signup Cohort
- March Signup Cohort
- April Signup Cohort
- May Signup Cohort
- June Signup Cohort

3. Define User Activity (Retention Criteria)

A user is considered retained in a given month if:

- The user performs at least one successful transaction during that month.

This ensures retention reflects actual app usage, not just app installation.

4. Data Required

To perform the cohort analysis, the following data is needed:

- User ID
- Signup Date
- Transaction Date

- Transaction Status (successful/failed)

5. Cohort Grouping and Tracking

Steps:

1. Assign each user to a signup cohort based on their signup month.
2. Track whether users from each cohort made transactions in:
    - Month 0 (signup month)
    - Month 1
    - Month 2
    - Month 3, and so on
3. Count the number of active users per cohort per month.

6. Retention Metric

Monthly Retention Rate Formula:

$$\text{Monthly Retention Rate} = \frac{\text{Number of Active Users in Month N}}{\text{Total Users in Signup Cohort}} \times 100$$

Where:

- Month 0 = Signup month
- Month N = Subsequent months after signup

7. Cohort Table Representation

The results can be visualized using a cohort retention table:

Signup Month	Month 0	Month 1	Month 2	Month 3
January	100%	65%	48%	40%
February	100%	62%	45%	38%
March	100%	68%	52%	44%

This table highlights how retention declines over time for each cohort.

8. Analysis and Insights

Using the cohort analysis, the company can:

- Identify which signup months have higher or lower retention
- Understand how quickly users drop off after signup
- Detect improvements or declines in engagement over time
- Evaluate the effectiveness of onboarding, rewards, or feature changes

9. Business Use Case

Insights from this cohort analysis can help the company:

- Improve onboarding experiences
- Optimize user engagement strategies
- Design targeted retention campaigns
- Measure the long-term success of product updates or marketing efforts

Conclusion

By grouping users based on their signup month and tracking their transactional activity over time, cohort analysis provides a clear and structured view of user retention. This approach enables the fintech company to measure engagement trends, identify weak retention periods, and make data-driven decisions to improve long-term user retention and product performance.

Question 2: Why does retention drop after the first month, and how can it be improved?

Possible Reasons for Retention Drop:

1. **Weak Onboarding Experience**
   - Users may not clearly understand the app's value after the first transaction.
2. **No Immediate Incentive to Return**
   - Users complete one payment but see no reason to come back.
3. **Low Engagement Triggers**
   - Lack of reminders, notifications, or personalized messages.
4. **Limited Use Cases**
   - Users may only need the app once a month (e.g., bill payment).
5. **Trust or UX Issues**
   - Concerns around security, failed transactions, or slow support.

Strategies to Address Retention Drop-Off:

1. **Improve Onboarding**
   - Guide users to complete a second transaction.
   - Show benefits like rewards, savings, or credit insights.
2. **Second-Transaction Incentives**
   - Offer cashback or rewards for completing the second payment.
3. **Personalized Notifications**
   - Send reminders for bill due dates or unused accounts.
   - Use behaviour-based nudges instead of generic messages.
4. **Expand Engagement Touchpoints**
   - Introduce additional features like rewards tracking, credit score updates, or offers.
5. **Customer Feedback Loop**
   - Collect feedback from inactive users and fix recurring issues.

Conclusion

Cohort analysis helps identify when and why users drop off. If retention declines after the first month, it usually indicates low perceived value or engagement. By improving onboarding, personalizing communication, and incentivizing repeat usage, the company can significantly improve retention and long-term user value.

Scenario 2:

The fintech company is testing two different loan approval notification designs. Version A is a simple approval message, while Version B includes additional loan details (e.g., repayment options, interest rate, and payment reminders). They want to see which design leads to more loan acceptance.

Question 1: How would you structure an A/B test to measure the impact of these notification designs on loan acceptance rates?

Structuring an A/B Test to Measure Impact of Notification Designs on Loan Acceptance Rates

To measure the impact of two different loan approval notification designs on loan acceptance rates, a structured A/B testing framework should be followed to ensure accurate, unbiased, and actionable results.

1. Objective of the A/B Test

The primary objective is to determine whether detailed loan approval notifications (Version B) lead to higher loan acceptance rates compared to simple approval notifications (Version A).

2. Define Variants

Variant	Description
Version A (Control)	Simple loan approval message with minimal information
Version B (Variant)	Detailed notification including loan amount, interest rate, repayment tenure, EMI details, and reminders

3. Hypothesis

- Null Hypothesis (H_0): There is no significant difference in loan acceptance rates between Version A and Version B.
- Alternative Hypothesis (H_1): Version B (detailed notification) results in a higher loan acceptance rate than Version A.

4. User Segmentation & Random Assignment

- Select eligible users who receive loan approval notifications.
- Randomly split users into two equal and statistically comparable groups:
  - Group A → Version A
  - Group B → Version B
- Ensure isolation so that users see only one version during the experiment.

5. Metrics to Track

Primary Metric

- Loan Acceptance Rate

$$\text{Loan Acceptance Rate} = \frac{\text{Loans Accepted}}{\text{Loans Offered}} \times 100$$

Secondary Metrics

- Average Loan Amount Accepted
- Repayment Behaviour
  - On-time repayment rate
  - Early repayment rate
  - Default rate
- Time to Accept Loan
- Customer Support Queries (Post-Notification)

6. Experiment Duration

- Run the experiment for a fixed time period (e.g., 2–4 weeks) or until:
  - Statistically significant sample size is achieved
  - Seasonal or promotional bias is minimized

7. Data Collection

Collect and store data for both variants including:

- Number of users shown the notification
- Number of accepted loans
- Loan amount accepted
- Repayment behaviour over the first billing cycle

8. Analysis

- Compare acceptance rates between Version A and Version B.
- Use statistical tests (e.g., Z-test or Chi-square test) to validate significance.
- Analyse secondary metrics to ensure higher acceptance does not increase default risk.

9. Decision Criteria

- If Version B shows a statistically significant increase in loan acceptance without worsening repayment behaviour, it should be considered the better-performing design.
- If repayment defaults increase, further refinement or partial information disclosure may be required.

Conclusion

By structuring the A/B test with clear objectives, controlled experimentation, and well-defined metrics, the fintech company can confidently assess the impact of notification design on loan acceptance rates and make data-driven product decisions.

Question 2: How to Evaluate Whether a 10% Increase in Loan Acceptance Justifies Additional Complexity

Evaluating the Business Viability of Version B

When Version B leads to a measurable uplift in loan acceptance but requires additional implementation effort, the evaluation must be structured around financial impact, operational feasibility, and long-term strategic value.

1. Objective of the Evaluation

The core objective is to determine whether the incremental business value generated by the higher loan acceptance rate outweighs the total technical, operational, and financial costs associated with deploying Version B.

This ensures that the decision supports profitability and sustainable growth rather than short-term metric improvement.

2. Define the Performance Uplift

Clearly quantify the improvement observed in experimentation:

- Measure baseline acceptance rate under Version A
- Measure acceptance rate under Version B
- Calculate absolute and relative improvement
- Validate statistical significance

This confirms that the 10% uplift is reliable and not due to random variation.

3. Estimate Incremental Revenue Impact

Translate the uplift into financial value.

Required inputs:

- Total loan applications per month
- Increase in approved loans due to uplift
- Average revenue per approved loan (interest margin, fees, cross-sell potential)

Incremental Revenue = Additional Approved Loans × Average Revenue per Loan

This step converts performance improvement into tangible business gain.

4. Identify Implementation and Operational Costs

Evaluate all cost components:

Development Costs

- Engineering effort
- Product design modifications
- Testing and deployment resources

Operational Costs

- Infrastructure or system load
- Ongoing maintenance
- Customer support impact
- Compliance or regulatory adjustments

Total Cost = One-Time Investment + Recurring Costs

5. Conduct Profitability and ROI Assessment

Determine financial justification:

Net Benefit = Incremental Revenue − Total Cost

Return on Investment (ROI) = (Net Benefit / Total Cost) × 100

If ROI meets or exceeds internal benchmarks, implementation is economically justified.

6. Analyze Risk and Credit Quality Impact

Higher acceptance must not compromise portfolio quality.

Assess:

- Change in default rates
- Risk-adjusted return on loans
- Impact on Non-Performing Assets (NPA)
- Credit risk profile of newly accepted borrowers

If acceptance increases but credit quality deteriorates, long-term profitability may decline.

7. Evaluate Operational Complexity and Scalability

Consider broader system implications:

- Does the feature increase technical debt?
- Will it slow down future releases?
- Is the system scalable with growing traffic?
- Does it increase regulatory scrutiny?

Operational burden must not outweigh incremental gains.

8. Explore Optimization Alternatives

If the cost-benefit balance is marginal:

- Identify which components of the detailed notification drive the uplift

- Remove low-impact elements
- Test simplified variants through additional experimentation

This approach maximizes value while minimizing complexity.

9. Strategic Alignment Assessment

Ensure Version B aligns with:

- Long-term product roadmap
- Customer transparency goals
- Brand positioning
- Competitive differentiation

A feature delivering both financial and strategic value strengthens justification.

Conclusion

A 10% increase in loan acceptance is beneficial only if it generates sustainable incremental revenue that exceeds the total cost of implementation and associated risks. By systematically evaluating financial returns, credit risk implications, operational complexity, and strategic alignment, the organization can determine whether Version B delivers meaningful long-term business value or requires optimization before full-scale rollout.

DBMS Scenario Question

1) ACID Properties

Q1. Fund transfer crashes midway — prevent partial corruption (Atomicity + Durability)

Goal: Either *both* balances update and ledger writes happen, or *nothing* happens.

Approach

- Wrap everything in a single DB transaction.
- Lock both accounts deterministically to avoid deadlocks.
- Use double-entry ledger + compute balances from ledger (or keep account_balances as a cached projection updated atomically).
- Commit only at the end. If a crash happens before commit DB rolls back.

Durability

- Use normal WAL durability (Postgres default). If you need "stronger" guarantees: ensure synchronous_commit=on for critical flows.

Extra production hardening

- Add an idempotency_key column to transactions to avoid double-processing retries.
- Enforce invariants with constraints/triggers (e.g., available_balance >= 0).

Q2. Two traders place orders nearly same time — avoid interference (Isolation + Consistency)

Goal: Each transaction sees a correct view and doesn't overwrite the other.

Options

1. Row-level locking (SELECT ... FOR UPDATE) on the row being changed.
2. Serializable isolation for strongest correctness (may abort one txn → retry).
3. Optimistic concurrency with version columns (CAS updates).

Example: preventing double-spend on same account

Why it works

- FOR UPDATE prevents two txns from changing the same balance simultaneously.
- SERIALIZABLE prevents anomalies under complex concurrent patterns.

2) Indexing

Q3. Real-time dashboards for balances + transactions — how indexing helps

Dashboards usually filter by:

- user_id, account_id
- recency (initiated_at / completed_at)
- status

Result: fewer scans, faster "latest X transactions" and "all accounts of user" queries.

Q4. Transaction table updated + queried by txn_id and date — best indexing trade-offs

- Each extra index slows inserts/updates.
- Prefer 1–2 high-value composite indexes over many single-column indexes.
- Consider partial indexes for "hot subsets":

3) Partitioning

Q5. Billions of transactions across years — efficient querying + archiving

Use range partitioning by time (monthly/weekly partitions). Queries for recent months hit only recent partitions.

- Detach old partitions and move to cold storage:
  - ALTER TABLE ... DETACH PARTITION ...
  - dump partition to object storage / archive DB.

Q6. Data residency (region-specific storage) — partitioning approach

Use partitioning by region (LIST partitioning) using a region_code on user or transaction rows.

Example:

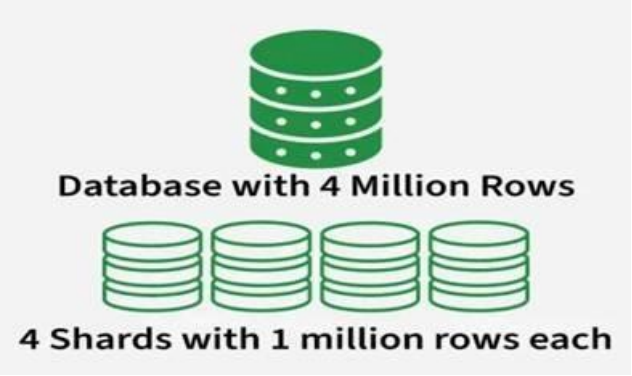
- Place partitions on region-specific infrastructure / clusters.
- Makes compliance audits simpler ("all EU data lives here").

4) Sharding

Q7. Peak hours microtransactions — horizontal sharding design

Sharding = split data across multiple DB nodes.

shard = hash(user_id) % N



What to shard

- High-write tables: transactions, ledger_entries (and possibly account_balances if truly massive)

Pattern

- Route writes/reads to shard based on user_id.
- Keep small reference tables centralized if needed.

Q8. Sharded by customer_id but need global analytics — what problems?

Main issues

- Cross-shard joins are expensive (need scatter-gather).
- Global aggregates (total volume, fraud patterns) require querying every shard.
- Harder to enforce global constraints (e.g., unique across all shards).
- Operational complexity (rebalancing shards, hot-shard when some customers are very active).

Typical fix

- Maintain a separate analytics warehouse (or HTAP replica) where data is consolidated for global queries.

5) Normalization vs Denormalization

Q9. Customer profiles + accounts + KYC docs — how normalization helps

Your schema is already nicely normalized:

- users holds identity
- accounts holds user financial containers
- user_kyc holds verification records
- kyc_documents holds document metadata

Benefits

- Avoids duplication (one user, many KYC docs).
- Prevents update anomalies (email changes once).
- Enforces integrity via FKs (kyc_documents.kyc_id → user_kyc.kyc_id).

Q10. Monthly reporting — normalized or denormalized?

Prefer: hybrid

- Keep OLTP normalized (your current schema).
- Build denormalized reporting tables/materialized views for fast month-end reporting.

This avoids heavy joins/scans on billions of OLTP rows during reporting.

6) Data Lakes vs Data Warehouses

Q11. Where store unstructured logs, clickstream, structured accounts?

- Data Lake (S3/GCS/HDFS): raw logs, clickstream, app events (semi/unstructured, schema evolves).
- Warehouse (Snowflake/BigQuery/Redshift): cleaned, modeled tables (facts/dimensions) for BI/analytics.
- OLTP DB (Postgres/MySQL): your live accounts, balances, transactional state.

Q12. How warehouse supports fraud detection + risk analysis

- Warehouse stores long history cheaply and queryable.
- Supports OLAP: cohort analysis, trends, segment risk, anomaly baselines.
- Enables feature generation:
    - rolling 7/30/90-day spend
    - merchant/category velocity
    - chargeback rates
    - geo/device inconsistencies

7) Data Ingestion and Pipelines

Q13. Ingest real-time payments from mobile apps for fraud monitoring

Streaming pipeline

1. Mobile app → API gateway
2. Payments service writes to OLTP + emits event to Kafka/Kinesis (payment_initiated, payment_completed)
3. Stream processor (Flink/Spark Streaming) computes risk signals in near real-time
4. Write fraud outputs to:

    - fraud_alerts table for operational actions
    - feature store / warehouse for model training

Q14. Import credit scores daily — ingestion approach

Batch ingestion

- Scheduled job (Airflow/Prefect/cron)
- Pull from provider SFTP/API
- Land raw into lake ("bronze")
- Validate + transform to "silver"
- Load into warehouse + optionally OLTP cache table

Key needs:

- schema validation
- retries
- idempotency (same day file shouldn't double load)

8) ETL

Q15. ETL pipeline for investment portfolios from multiple banks

Design

- **Extract:** pull bank files/APIs into lake raw zone.
- **Transform:**
    - normalize identifiers (PAN hash / phone hash / internal user_id mapping)
    - deduplicate (same holding from two feeds)
    - standardize instruments (ISIN mapping)
    - currency conversion rates
- **Load:**
    - warehouse fact tables (holdings, positions, valuations)
    - dimension tables (user, instrument, broker)

Q16. ETL failed after partial load — consistency + safe resume

Use:

- Staging tables + atomic swap
- Checkpointing + idempotent loads
- Upserts with deterministic keys

Pattern

1. Load into staging_portfolio_2026_02_11
2. Validate row counts, checksums
3. In a transaction:
    - DELETE/UPSERT into target partition for that date
    - mark job run as complete in etl_job_runs
4. If a job crashes, rerun safely because keys are deterministic.

9) Additional DBMS Topics

Q17. Regulator wants complete audit trail for loan applications

Use:

- append-only audit_trail (you already have it)
- write audits via triggers or application middleware
- store "before/after" snapshots (JSON) if needed
- make audit immutable: restrict updates/deletes, or store in WORM storage

Q18. ML model needs daily updated training data on loan defaults

- Daily incremental ETL from OLTP → warehouse feature tables
- Keep snapshots (partitioned by date) for reproducible training

Example:

- loan_repayments and loans → daily partition features_loan_default_dt=YYYY-MM-DD

Q19. Detect + resolve duplicates in credit card applications from multiple sources

In transform:

- Standardize fields (phone/email casing, remove punctuation)
- Use deterministic keys:
    - email_hash, phone_hash, maybe kyc_number_hash
- Fuzzy matching for names + DOB (if available)

Example logic:

- if same email OR same phone OR same kyc_number_hash → probable duplicate
- if close name match + same DOB → duplicate

Resolution:

- pick “golden record” and merge; retain lineage metadata.

Q20. Real-time analytics dashboards for txns, credit scores, investments — proposed DBMS architecture

- OLTP: Postgres for live state + strong consistency.
- Streaming bus: Kafka/Kinesis for events.
- Real-time serving:
    - Materialized views / OLAP store (ClickHouse/Druid/Pinot) for dashboards
    - Redis cache for hot aggregates (per-user recent spend, balances snapshot)
- Warehouse: for deep historical analytics, model training, compliance reports.

Why

1. OLTP stays fast for writes.
2. Dashboards don’t overload OLTP.
3. Analytics scales independently.